FinTech Nexus Database Architecture & Auditing

Author: Nithish
Role: Lead Data Analyst
Technology Stack: MySQL
Objective: To design, populate, secure, and audit a relational database for a simulated financial technology platform, demonstrating end-to-end database lifecycle management.

Executive Summary

Project FinTech Nexus showcases the comprehensive development of a secure relational database. The project bypassed pre-cleaned datasets, requiring the creation of strict architectural constraints, active record management (CRUD) for fraud prevention, and advanced relational auditing using a variety of SQL joins to extract hidden business insights.

Phase 1: Database Architecture & Security Constraints

The Challenge: The Chief Information Security Officer (CISO) required a strict hierarchy between users and their financial transactions. Invalid data (like negative transfer amounts or missing names) had to be blocked at the database engine level.

The Solution: Built a parent (App_Users) and child (Transactions) table structure linked by a FOREIGN KEY. Applied robust constraints including PRIMARY KEY, NOT NULL, UNIQUE, CHECK (to prevent negative amounts), and DEFAULT values.

SQL

```
CREATE DATABASE Architecture;
USE Architecture;

-- Parent Table
CREATE TABLE App_Users(
    User_ID INT PRIMARY KEY,
    Full_Name VARCHAR(50) NOT NULL,
    Email VARCHAR(50) UNIQUE,
    Account_Status VARCHAR(50) DEFAULT 'Active'
);

-- Child Table
CREATE TABLE Transactions(
    Transaction_ID VARCHAR(50) PRIMARY KEY,
    Amount INT CHECK (Amount > 0),
    Transaction_date DATE DEFAULT (CURRENT_DATE),
    Sender_ID INT,
    FOREIGN KEY (Sender_ID) REFERENCES App_Users(User_ID)
);
```

Phase 2: Data Ingestion & Population

The Challenge: Populate the newly architected tables with initial user profiles and transaction records while maintaining referential integrity.

The Solution: Executed precise INSERT INTO statements, successfully mapping the Sender_ID in the Transactions table to the correct User_ID in the App_Users table. Intentionally included a dormant user (Charlie Brown) to test advanced joins in later phases.

SQL

```
INSERT INTO App_Users (User_ID, Full_Name, Email, Account_Status) VALUES
(1, 'Alice Smith', 'alice@email.com', DEFAULT),
(2, 'Bob Jones', 'bob@email.com', 'Suspended'),
(3, 'Charlie Brown', 'charlie@email.com', DEFAULT),
(4, 'Nithish', 'nithish@nexus.com', DEFAULT);

INSERT INTO Transactions (Transaction_ID, Amount, Sender_ID) VALUES
('TXN-1001', 500, 1),
('TXN-1002', 150, 2),
('TXN-1003', 1000, 4),
('TXN-1004', 50, 1);
```

Phase 3: Active Record Management & Security Audits

The Challenge: Act as a Risk & Compliance Analyst to quarantine a compromised account, process a fraud chargeback by removing a specific transaction, and generate a report of high-value transactions.

The Solution: Utilized targeted UPDATE and DELETE commands using Primary Keys to ensure safe data manipulation. Executed an INNER JOIN paired with a WHERE clause to filter for high-value transfers.

SQL

```
-- Fraud Alert: Quarantining an account
UPDATE App_Users
SET Account_Status = 'Suspended'
WHERE User_ID = 1;

-- Chargeback: Deleting a fraudulent record safely
DELETE FROM Transactions
WHERE Transaction_ID = 'TXN-1004';

-- High-Value Audit: Filtering joined tables
SELECT
    App_Users.Full_Name,
    Transactions.Amount
FROM App_Users
INNER JOIN Transactions
    ON App_Users.User_ID = Transactions.Sender_ID
WHERE Transactions.Amount > 200;
```

Phase 4: Advanced Extraction & Formatting

The Challenge: Fulfill specific departmental requests: segmenting external marketing emails, identifying the platform's single largest transaction, and formatting output for an automated billing system.

The Solution: Leveraged pattern matching (LIKE '%email.com'), advanced sorting (ORDER BY DESC LIMIT 1), and string manipulation functions (UPPER()) to format and extract precise business intelligence.

SQL

```
-- Email Domain Filter
SELECT Full_Name, Email
FROM App_Users
WHERE Email LIKE '%email.com';

-- The Big Spender Filter
SELECT Transaction_ID, Amount
FROM Transactions
ORDER BY Amount DESC
LIMIT 1;

-- Billing System Formatting
SELECT UPPER(Full_Name)
FROM App_Users;
```

Phase 5: The Master Joiner (Advanced Relational Mapping)

The Challenge: Move beyond simple data matches to uncover hidden system insights, specifically identifying registered users with zero transactions and running stress-test simulations on suspended accounts.

The Solution: Bypassed INNER JOIN logic to utilize non-matching relational maps. Used LEFT JOIN and RIGHT JOIN to intentionally generate NULL values that exposed dormant accounts. Executed a CROSS JOIN to multiply tables together for a risk simulation matrix.

SQL

```
-- Dormant User Audit (Exposes users with no transactions)
SELECT
    App_Users.Full_Name,
    Transactions.Transaction_ID
FROM App_Users
LEFT JOIN Transactions
    ON App_Users.User_ID = Transactions.Sender_ID;

-- Reverse Engineered Audit
SELECT
    Transactions.Transaction_ID,
    App_Users.Full_Name
FROM Transactions
RIGHT JOIN App_Users
    ON Transactions.Sender_ID = App_Users.User_ID;

-- Suspended Risk Matrix Simulation
SELECT
    App_Users.Full_Name,
    Transactions.Transaction_ID
FROM App_Users
CROSS JOIN Transactions
WHERE App_Users.Account_Status = 'Suspended';
```

End of Report